Lab 4 — MPI Lattice Boltzmann

Author: Patron Antoine **Collaboration:** Solo work (no teammate)

---

1. Overview

This lab implements an MPI communication layer on top of a provided D2Q9 Lattice Boltzmann solver. The computational kernel (collision, propagation, boundary conditions) was already supplied; the work consisted in splitting the global mesh across MPI processes and exchanging ghost cells between neighbors.

All six non-optional exercises have been implemented and validated. The optional exercise 7 (Grid'5000 scalability study) was not performed.

No known bug.

---

2. Validation methodology

Three checks were run for each exercise N (from 1 to 6):

1. **Per-cell position check** — every ghost cell is filled with a unique integer depending on its absolute position, so the check catches any wrong source, wrong destination, or wrong offset:

   ```
   mpirun -np 4 ./check_comm -e N -p position -s both
   ```

2. **Modulo-9 pattern** — shorter numbers that keep the terminal output readable; useful to spot off-by-one bugs by visual inspection:

   ```
   mpirun -np 4 ./check_comm -e N -p modulo9 -s both
   ```

3. **Checksum against the sequential reference** — runs the full simulation and compares the rendered frame at step 3 against the one produced by exercise 0:

   ```
   # reference
   mpirun -np 4 ./lbm -e 0
   ./display --gnuplot output.raw 3 | md5sum > ref.md5

   # check
   mpirun -np 4 ./lbm -e N
   ./display --gnuplot output.raw 3 | md5sum -c ref.md5
   ```

The three checks succeed for every exercise from 1 to 6.

---

3. Exercise 1 — 1D blocking communication

What was done

- Filled `lbm_comm_init_ex1` to split the global mesh along the X axis only: `nb_x = comm_size`, `nb_y = 1`, and each rank owns a sub-domain of `(total_width / nb_x) + 2` columns (the `+ 2` accounts for the two ghost columns).
- The program aborts via `fatal(...)` when `total_width` is not a multiple of the number of MPI processes, as required.
- Implemented `lbm_comm_ghost_exchange_ex1` using `MPI_Send` / `MPI_Recv`, with left/right neighbors set to `MPI_PROC_NULL` at the domain borders.

Design choices

- **Using `MPI_PROC_NULL` for borders.** Instead of branching on edge cases, the neighbor is set to `MPI_PROC_NULL` at the global mesh border. The MPI standard guarantees that sends to, and receives from, `MPI_PROC_NULL` are no-ops. This keeps the exchange function linear and branch-free.
- **Send-right-then-recv-left ordering.** A purely symmetric send-then-receive on every rank would deadlock in strict mode. The chosen pattern — send to the right, then receive from the left — breaks the cycle because the receive on rank k matches the send from rank k−1 that happens to already be in flight.
- **Two tags.** A dedicated tag per direction (`TAG_TO_RIGHT` and `TAG_TO_LEFT`) removes any ambiguity if the MPI implementation chooses to reorder messages.

Known bugs

None.

---

4. Exercise 2 — 1D odd/even ordering

What was done

- `lbm_comm_init_ex2` reuses `lbm_comm_init_ex1` verbatim — the splitting is identical.
- `lbm_comm_ghost_exchange_ex2` changes only the ordering: even ranks receive first and send second, odd ranks send first and receive second.

Rationale

The naive pattern of exercise 1 can serialize communications when the MPI implementation does not buffer sends eagerly: each rank waits for its left neighbor before proceeding. The odd/even pattern pairs ranks two-by-two so that all pairs can exchange in parallel, cutting the theoretical latency of a linear chain from O(P) to O(1) for a given direction.

Known bugs

None.

---

5. Exercise 3 — 1D non-blocking communication

What was done

- Same init as ex1.
- `lbm_comm_ghost_exchange_ex3` posts the two `MPI_Irecv` calls first, then the two `MPI_Isend` calls, and terminates with a single `MPI_Waitall(4, ...)`.

Design choices

- **Receives posted before sends.** Posting `MPI_Irecv` before the matching `MPI_Isend` allows the MPI runtime to place incoming data directly into the user buffer, bypassing the implementation's internal buffering. This is a standard optimization when the receive buffer is known.
- **Single `MPI_Waitall` at the end.** All four requests are released in one call, which keeps the function synchronous from the caller's point of view (i.e., ghost cells are guaranteed to be up to date when the function returns) while letting the runtime overlap the four transfers freely.

Known bugs

None.

---

6. Exercise 4 — 2D splitting with manual copy

What was done

- `lbm_comm_init_ex4` uses `MPI_Dims_create` to pick a balanced 2D process grid, creates a non-periodic Cartesian communicator with `MPI_Cart_create`, and stores it in `comm->communicator`. The rank coordinates are recovered with `MPI_Cart_coords`.
- Allocation of the four send/receive buffers (`buffer_send_up`, `buffer_send_down`, `buffer_recv_up`, `buffer_recv_down`) happens once in init; the buffers are freed in `lbm_comm_release_ex4`.
- `lbm_comm_ghost_exchange_ex4` handles the 8 neighbors by grouping them into three families:
  1. Left/right exchange with contiguous `H × 9` doubles using `MPI_Sendrecv`. This implicitly carries the corners (since the ghost columns include the corner cells).
  2. Up/down exchange with manual packing into the temporary buffers, then `MPI_Sendrecv`, then manual unpacking into the ghost rows. This is required because rows are **not** contiguous in memory due to the column-major layout.
  3. The four corner cells (`DIRECTIONS` doubles each) are exchanged with four additional `MPI_Sendrecv` calls. *(Note: with the implicit corner transfer via the horizontal send, these four diagonal exchanges are technically redundant in ex4; they are kept for clarity and because `check_comm` verifies the diagonal transfers independently.)*

Design choices

- **Helper `lbm_neighbor_rank`.** A single static function computes the neighbor rank from an `(rank_x + dx, rank_y + dy)` coordinate pair, returning `MPI_PROC_NULL` if the coordinate is outside the grid. This centralizes the corner-case logic and makes the exchange function linear and symmetric in all 8 directions.
- **Pre-allocated buffers.** The four exchange buffers are allocated in init and reused at every time step. No allocation happens on the hot path.
- **`MPI_Sendrecv` rather than send/recv pairs.** `MPI_Sendrecv` is deadlock-free by construction and removes the need for the odd/even or non-blocking patterns of the previous exercises.

Known bugs

None.

---

7. Exercise 5 — 2D with MPI datatypes

What was done

- Same init as ex4, plus the creation of an `MPI_Type_vector` stored in `comm->type`. The type is committed in init and freed in release.
- `lbm_comm_ghost_exchange_ex5` replaces the manual pack/unpack loops of ex4 by a single `MPI_Sendrecv` per direction using `comm->type`. The horizontal (left/right) exchange is unchanged, as it was already contiguous.

Design choices

- **`MPI_Type_vector` parameters.** The type describes one ghost row (top or bottom) as:

  ```
  count       = comm->width                 // number of cells in a row
  blocklength = DIRECTIONS                  // 9 doubles per cell
  stride      = comm->height * DIRECTIONS   // distance (in doubles) between
                                            // two cells in the same row
  oldtype     = MPI_DOUBLE
  ```

  The `stride` is the key piece: because the mesh is stored column-major, two cells at positions `(x, y)` and `(x + 1, y)` are separated by exactly `comm->height * DIRECTIONS` doubles in memory. The MPI_Type_vector picks one cell every stride, `comm->width` times — exactly describing a ghost row.

- **Corners handled separately.** The four corner exchanges still use `MPI_DOUBLE` (9 doubles per corner) because creating a type for a single cell would bring no benefit.

Known bugs

None.

---

8. Exercise 6 — 2D non-blocking

What was done

- Init and release inherit directly from ex5 (same Cartesian communicator and same MPI datatype).
- `lbm_comm_ghost_exchange_ex6` performs the exchange in two batches:
  1. **Cardinal directions (left, right, up, down).** Four `MPI_Irecv` then four `MPI_Isend`, finalized by a single `MPI_Waitall(8, ...)`.
  2. **Diagonal directions (up-left, up-right, down-left, down-right).** Another four `MPI_Irecv` + four `MPI_Isend` on the same request array, finalized by a second `MPI_Waitall(8, ...)`.

Design choices

- **Two batches instead of one.** The lab subject warns that when corners are carried both by the horizontal and by the diagonal sends, a corner cell can be overwritten twice during the same exchange. Running the cardinal exchange to completion before starting the diagonal exchange avoids that race by making sure the cardinal writes to the corner are visible before the diagonal `Irecv` overwrites them.
- **Crossed tags on the diagonals.** For each diagonal pair, the send tag is the opposite of the receive tag so that a send to `up_left` (tagged 7) matches a receive posted by the neighbor from `down_right` (also tagged 7, but on the other side this is their `down_right` tag).
- **Request array reused.** The 8 requests in `comm->requests` are reused between the two batches because the first `MPI_Waitall` completes them all.

Known bugs

None.

---

9. Cross-cutting design choices

A handful of patterns are reused across all six exercises:

- **`MPI_PROC_NULL` at every border**, removing branching inside the communication routines.
- **Helper `lbm_neighbor_rank` / `lbm_neighbor_rank_ex5` / `lbm_neighbor_rank_ex6`** centralising the out-of-bounds logic. (Three copies exist because each exercise lives in its own translation unit; the logic is identical.)
- **Receives posted before sends** in the non-blocking variants.
- **Pre-allocated exchange buffers** in the 2D manual variant, no allocation on the hot path.
- **Separate tags per direction** to disambiguate messages — even though the Cartesian communicator would in principle let us rely on ordering alone, explicit tags make debugging with `check_comm` much easier.

---

10. What was not done

Exercise 7 — scalability on Grid'5000 was not performed. A natural next step would be to run the provided `bench_correction.sh` script on a mesh large enough to amortize startup costs, and to compare:

- strong scaling of ex1 (linear chain) against ex2 (odd/even) and ex3 (non-blocking) to quantify the serialization cost of the naive 1D pattern;
- strong scaling of ex4, ex5 and ex6 at large process counts where the 2D splitting reduces the perimeter-to-area ratio of each sub-domain compared with the 1D splitting.

A weak-scaling study using the `--scaling` option would also be informative since it would reveal the cost of the MPI datatype machinery in ex5 relative to the manual copy of ex4.